

Segurança na camada de transporte

17

TÓPICOS ABORDADOS

17.1 CONSIDERAÇÕES DE SEGURANÇA NA WEB

Ameaças à segurança na Web
Técnicas de segurança de tráfego Web

17.2 SECURE SOCKETS LAYER

Arquitetura SSL
Protocolo de registro SSL
Protocolo de especificação de mudança de cifra
Protocolo de alerta
Protocolo de handshake
Cálculos criptográficos

17.3 TRANSPORT LAYER SECURITY

Número de versão
Message Authentication Code
Função pseudoaleatória
Códigos de alerta
Conjuntos de cifras
Tipos de certificado do cliente
`Certificate_verify` e mensagens de conclusão
Cálculos criptográficos
Preenchimento

17.4 HTTPS

Início de conexão
Fechamento de conexão

17.5 SECURE SHELL (SSH)

Protocolo da Camada de Transporte
Protocolo de Autenticação de Usuário
Protocolo de Conexão

17.6 LEITURA RECOMENDADA

17.7 PRINCIPAIS TERMOS, PERGUNTAS PARA REVISÃO E PROBLEMAS

OBJETIVOS DE APRENDIZAGEM

APÓS ESTUDAR ESTE CAPÍTULO, VOCÊ SERÁ CAPAZ DE:

- ☒ Resumir as ameaças à segurança da rede e os métodos de segurança do tráfego na Web.
- ☒ Apresentar uma visão geral da Secure Socket Layer (SSL).
- ☒ Compreender as diferenças entre Secure Socket Layer e Transport Layer Security.
- ☒ Comparar a função pseudoaleatória usada na Transport Layer Security com aquelas discutidas anteriormente no livro.
- ☒ Apresentar uma visão geral do HTTPS (HTTP sobre SSL).
- ☒ Apresentar uma visão geral do Secure Shell (SSH).

"Não podemos fazer aliança com príncipes vizinhos até que tenhamos informações sobre seus projetos."

— *A Arte da Guerra*, Sun Tzu

Praticamente todas as empresas, a maioria das agências do governo, e muitos indivíduos, agora possuem Websites. O número de indivíduos e empresas com acesso à Internet está expandindo rapidamente e todas elas têm navegadores Web gráficos. Como resultado, as empresas estão entusiasmadas com a construção de facilidades na Web para o comércio eletrônico. Mas a realidade é que a Internet e a Web são extremamente vulneráveis a comprometimentos de vários tipos. À medida que as empresas acordam para essa realidade, a demanda por serviços Web seguros aumenta.

O assunto de segurança na Web é muito amplo e pode facilmente encher um livro inteiro. Neste capítulo, começamos com uma discussão sobre os requisitos gerais para a segurança na Web e depois focalizamos três esquemas padronizados que estão se tornando cada vez mais importantes como parte do comércio na Web: SSL/TLS; HTTPS e SSH.

17.1 CONSIDERAÇÕES SOBRE SEGURANÇA NA WEB

A World Wide Web é fundamentalmente uma aplicação cliente/servidor trabalhando sobre a Internet e intranets TCP/IP. Dessa forma, as ferramentas de segurança e as técnicas discutidas até aqui neste livro são relevantes à questão da segurança na Web. Porém, as seguintes características do uso da Web sugerem a necessidade de ferramentas de segurança apropriadas:

- Embora os navegadores Web sejam muito fáceis de usar, os servidores Web sejam relativamente fáceis de se configurar e gerenciar, e o conteúdo Web cada vez mais fácil de desenvolver, o software por trás disso é extraordinariamente complexo. Esse software complexo pode ocultar muitas falhas de segurança em potencial. A curta história da Web é repleta de exemplos de sistemas novos e atualizados, devidamente instalados, que são vulneráveis a uma série de ataques à segurança.
- Um servidor Web pode ser explorado como uma plataforma de lançamento para o complexo inteiro de computadores da empresa ou da agência. Quando o servidor Web é comprometido, um atacante pode ser capaz de obter acesso a dados e sistemas que não fazem parte da própria Web, mas que estão conectados ao servidor no site local.
- Usuários casuais e não treinados (em questões de segurança) são clientes comuns para serviços baseados na Web. Esses usuários não estão necessariamente cientes dos riscos de segurança que existem e não possuem as ferramentas ou o conhecimento para tomar contramedidas eficazes.

Ameaças à segurança na Web

A Tabela 17.1 oferece um resumo dos tipos de ameaças à segurança enfrentadas no uso da Web. Uma forma de agrupar essas ameaças é em termos de ataques passivos e ativos. Ataques passivos incluem espionagem do tráfego da rede entre navegador e servidor, e obtenção de acesso a informações em um Website que deveria ser restrito. Ataques ativos incluem personificação de outro usuário, alteração de mensagens em trânsito entre cliente e servidor, e alteração de informações em um Website.

Outra forma de classificar as ameaças de segurança na Web é em termos do local da ameaça: servidor Web, navegador Web e tráfego de rede entre navegador e servidor. As questões de segurança de servidor e navegador entram na categoria de segurança do sistema de computador; a Parte Seis deste livro focaliza a questão de segurança do sistema em geral, mas também se aplica à segurança de sistemas Web. As questões de segurança de tráfego entram na categoria de segurança da rede e são focalizadas neste capítulo.

Tabela 17.1 Comparação de ameaças na Web.

	Ameaças	Consequências	Contramedidas
Integridade	■ Modificação de dados do usuário ■ Navegador cavalo de Tróia ■ Modificação de memória ■ Modificação de tráfego de mensagem em trânsito	■ Perda de informações ■ Comprometimento da máquina ■ Vulnerabilidade a todas as outras ameaças	Checksums criptográficos
Confidencialidade	■ Espionagem na rede ■ Roubo de informações do servidor ■ Roubo de dados do cliente ■ Informações sobre configuração de rede ■ Informações sobre qual cliente fala com o servidor	■ Perda de informações ■ Perda de privacidade	Criptografia, proxies Web
Negação de serviço	■ Encerramento de threads do usuário ■ Inundação da máquina com solicitações falsas ■ Preenchimento do disco ou da memória ■ Isolamento da máquina por ataques de DNS	■ Interrupção ■ Incômodo ■ Impede que usuário realize o trabalho	Difícil de impedir
Autenticação	■ Personificação de usuários legítimos ■ Falsificação de dados	■ Má representação do usuário ■ Crença de que informações falsas são válidas	Técnicas criptográficas

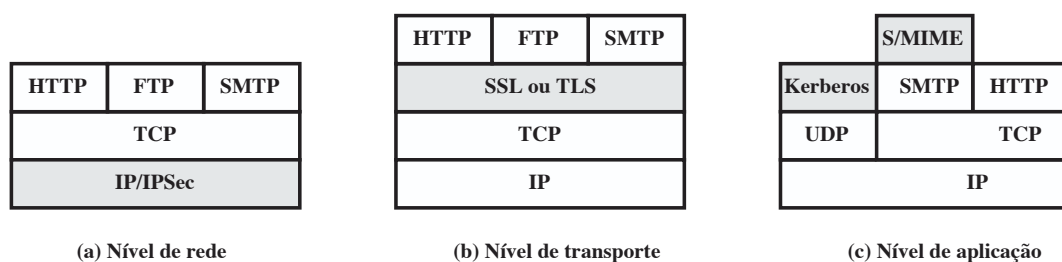
Técnicas de segurança de tráfego Web

Diversas técnicas para oferecer segurança na Web são possíveis. As várias técnicas que têm sido consideradas são semelhantes nos serviços que elas oferecem e, até certo ponto, nos mecanismos que elas utilizam, mas diferem com relação ao seu escopo de aplicabilidade e seu local relativo dentro da pilha de protocolos TCP/IP.

A Figura 17.1 ilustra essa diferença. Um modo de oferecer a segurança na Web é usar IP Security (IPSec) (Figura 17.1a). A vantagem de usar IPSec é que ele é transparente aos usuários finais e aplicações, e oferece uma solução de uso geral. Além disso, IPSec inclui uma capacidade de filtragem de modo que somente o tráfego selecionado precisa resultar em overhead do processamento do IPSec.

Outra solução relativamente de uso geral é implementar a segurança logo acima do TCP (Figura 17.1b).

Serviços de segurança específicos da aplicação são embutidos dentro da aplicação em particular. A Figura 17.1c mostra exemplos dessa arquitetura. A vantagem dessa técnica é que o serviço pode ser ajustado às necessidades específicas de determinada aplicação.

Figura 17.1 Local relativo das instalações de segurança na pilha de protocolos TCP/IP.

17.2 SECURE SOCKETS LAYER

Um dos serviços de segurança mais usados é o Secure Sockets Layer (SSL) e o consequente padrão da Internet, conhecido como Transport Layer Security (TLS), sendo este último definido na RFC 5246. SSL é um serviço de uso geral implementado como um conjunto de protocolos que contam com o TCP. Nesse nível, existem duas escolhas de implementação. Para a generalidade total, SSL (ou TLS) poderia ser fornecido como parte do conjunto de protocolos básico e, portanto, ser transparente às aplicações. Como alternativa, SSL pode ser embutido em pacotes específicos. Por exemplo, a maioria dos navegadores vem equipada com SSL, e a maioria dos servidores Web tem implementado o protocolo.

Esta seção é dedicada a uma discussão do SSLv3, e a próxima descreve as principais diferenças entre SSLv3 e TLS.

Arquitetura SSL

SSL é projetado para utilizar TCP para oferecer um serviço seguro confiável de ponta a ponta. SSL não é um protocolo isolado, mas duas camadas de protocolo, conforme ilustra a Figura 17.2.

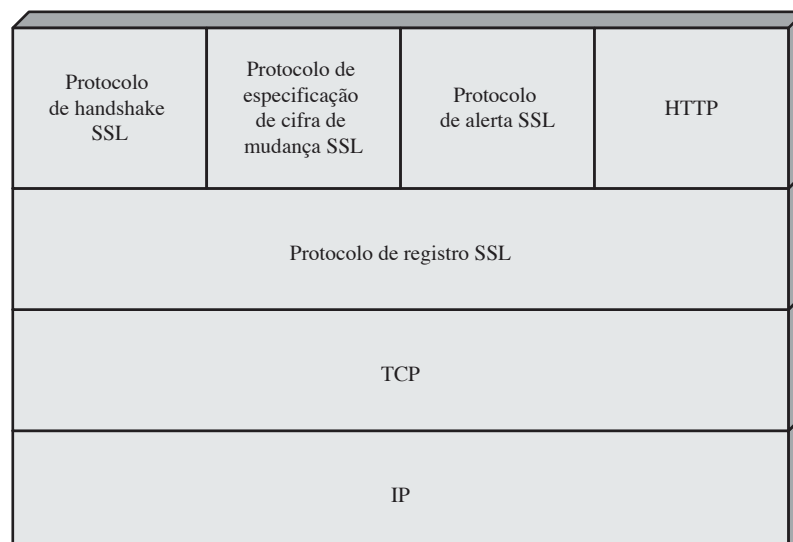
O protocolo de registro SSL oferece serviços básicos de segurança para vários protocolos de camada superior. Em particular, o Hypertext Transfer Protocol (HTTP), que oferece o serviço de transferência para interação cliente/servidor Web, pode operar em cima do SSL. Três protocolos de camada superior são definidos como parte do SSL: o protocolo de handshake (Handshake Protocol), o protocolo de especificação de mudança de cifra (Change Cipher Spec Protocol) e o protocolo de alerta (Alert Protocol). Esses protocolos específicos do SSL são usados no gerenciamento de trocas SSL e examinados mais adiante nesta seção.

Dois conceitos importantes do SSL são a sessão e a conexão SSL, que são definidas na especificação da seguinte forma:

- **Conexão:** uma conexão é um transporte (na definição do modelo de camadas OSI) que oferece um tipo adequado de serviço. Para SSL, essas conexões são relacionamentos par-a-par (peer-to-peer). As conexões são transientes. Cada conexão está associada a uma sessão.
- **Sessão:** uma sessão SSL é uma associação entre um cliente e um servidor. As sessões são criadas pelo protocolo de handshake. Elas definem um conjunto de parâmetros de segurança criptográficos, que podem ser compartilhados entre múltiplas conexões. As sessões são usadas para evitar a negociação dispendiosa de novos parâmetros de segurança para cada conexão.

Entre qualquer par de partes (aplicações como HTTP no cliente e servidor), pode haver múltiplas conexões seguras. Em teoria, também pode haver múltiplas sessões simultâneas entre as partes, mas esse recurso não é usado na prática.

Figura 17.2 Pilha de protocolos SSL.



Existem diversos estados associados a cada sessão. Quando uma sessão é estabelecida, existe um estado operacional atual para leitura e escrita (ou seja, recepção e envio). Além disso, durante o protocolo de handshake, são criados estados de leitura e escrita pendentes. Na conclusão bem-sucedida do protocolo de handshake, os estados pendentes se tornam os atuais.

Um estado de sessão é definido pelos seguintes parâmetros:

- **Identificador de sessão:** uma sequência de byte arbitrária, escolhida pelo servidor para identificar o estado de uma sessão ativa ou retomável.
- **Certificado do par:** um certificado X509.v3 do par. Esse elemento do estado pode ser nulo.
- **Método de compactação:** o algoritmo usado para compactar dados antes da encriptação.
- **Especificação de cifra:** especifica o algoritmo de encriptação de dados em massa (como null, AES etc.) e um algoritmo de hash (como MD5 ou SHA-1) usado para o cálculo do MAC. Ela também define os atributos criptográficos, como `hash_size`.
- **Segredo mestre:** segredo de 48 bytes compartilhado entre o cliente e o servidor.
- **É retomável:** um flag indicando se a sessão pode ser usada para iniciar novas conexões.

Um estado de conexão é definido pelos parâmetros a seguir.

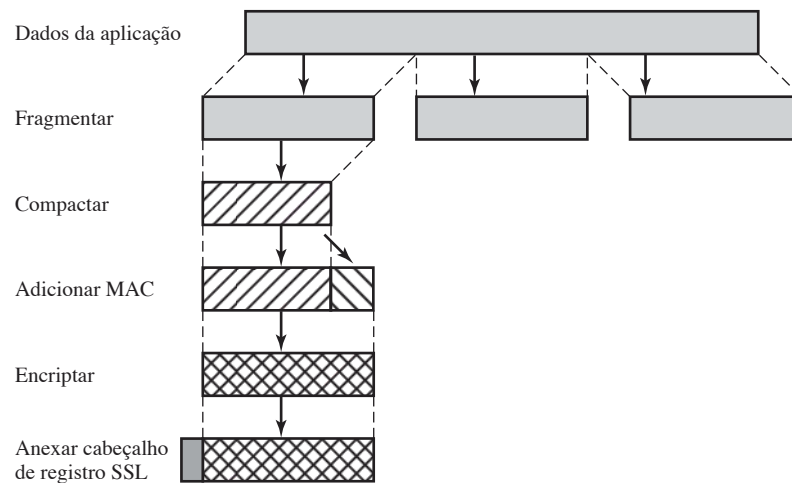
- **Aleatórios do servidor e cliente:** sequências de bytes que são escolhidas pelo servidor e cliente para cada conexão.
- **Segredo MAC de escrita do servidor:** a chave secreta usada nas operações MAC sobre dados enviados pelo servidor.
- **Segredo MAC de escrita do cliente:** a chave secreta usada em operações MAC sobre dados enviados pelo cliente.
- **Chave de escrita do servidor:** a chave de encriptação secreta para dados encriptados pelo servidor e decriptados pelo cliente.
- **Chave de escrita do cliente:** a chave de encriptação simétrica para dados encriptados pelo cliente e decriptados pelo servidor.
- **Vetores de inicialização:** quando uma cifra em bloco no modo CBC é usada, um vetor de inicialização (IV) é mantido para cada chave. Esse campo é primeiro inicializado pelo protocolo de handshake do SSL. Depois disso, o bloco de texto cifrado final de cada registro é preservado para uso como o IV com o registro seguinte.
- **Números de sequência:** cada parte mantém números de sequência separados para mensagens transmitidas e recebidas para cada conexão. Quando uma parte envia ou recebe uma mensagem de especificação de mudança de cifra o número de sequência apropriado é definido como zero. Números de sequência não podem exceder $2^{64} - 1$.

Protocolo de registro SSL

O protocolo de registro SSL oferece dois serviços para conexões SSL:

- **Confidencialidade:** o protocolo de handshake define uma chave secreta compartilhada que é usada para encriptação convencional de payloads SSL.
- **Integridade de mensagem:** o protocolo de handshake também define uma chave secreta compartilhada que é usada para formar um código de autenticação de mensagem (MAC, do acrônimo em inglês para *Message Authentication Code*).

A Figura 17.3 indica a operação geral do protocolo de registro SSL. Este recebe uma mensagem da aplicação a ser transmitida, fragmenta os dados em blocos de tamanho adequado, opcionalmente compacta os dados, aplica um MAC, encripta, acrescenta um cabeçalho e transmite a unidade resultante em um segmento TCP. Os dados recebidos são decriptados, verificados, descompactados e remontados e depois entregues a usuários de nível mais alto.

Figura 17.3 Operação do protocolo de registro SSL.

A primeira etapa é a **fragmentação**. Cada mensagem de camada superior é fragmentada em blocos de 2^{14} bytes (16384 bytes) ou menos. Em seguida, a **compactação** é aplicada opcionalmente. A compactação precisa ser sem perda e não pode aumentar o tamanho do conteúdo por mais do que 1024 bytes.¹ No SSLv3 (além da versão atual do TLS), nenhum algoritmo de compactação é especificado, de modo que o algoritmo de compactação default é nulo.

A etapa seguinte no processamento é calcular um **código de autenticação de mensagem** sobre os dados compactados. Para essa finalidade, é usada uma chave secreta compartilhada. O cálculo é definido como

```
hash(MAC_write_secret || pad_2 ||
      hash(MAC_write_secret || pad_1 || seq_num ||
            SSLCompressed.type || SSLCompressed.length ||
            SSLCompressed.fragment))
```

onde

	= concatenação
MAC_write_secret	= chave secreta compartilhada
hash	= algoritmo de hash criptográfico; ou MD5 ou SHA-1
pad_1	= o byte 0x36 (0011 0110) repetido 48 vezes (384 bits) para MD5 e 40 vezes (320 bits) para SHA-1
pad_2	= o byte 0x5C (0101 1100) repetido 48 vezes para MD5 e 40 vezes para SHA-1
seq_num	= o número de sequência para essa mensagem
SSLCompressed.type	= o protocolo de nível mais alto usado para processar esse fragmento
SSLCompressed.length	= o tamanho do fragmento compactado
SSLCompressed.fragment	= o fragmento compactado (se a compactação não for usada, o fragmento de texto claro)

Observe que isso é muito semelhante ao algoritmo HMAC definido no Capítulo 12. A diferença é que os dois preenchimentos são concatenados no SSLv3 e passam por um XOR no HMAC. O algoritmo MAC do SSLv3 é baseado no Internet draft original para HMAC, que usava concatenação. A versão final do HMAC, definida na RFC 2104, utiliza o XOR.

¹ Naturalmente, espera-se que a compactação encurte ao invés de expandir os dados. Porém, para blocos muito curtos, é possível, por conta de convenções de formatação, que o algoritmo de compactação por fim ofereça uma saída maior que a entrada.

Em seguida, a mensagem compactada mais o MAC são **encriptados** usando a encriptação simétrica. A encriptação não pode aumentar o tamanho do conteúdo por mais de 1024 bytes, de modo que o tamanho total não pode exceder $2^{14} + 2048$. Os algoritmos de encriptação a seguir são permitidos:

Cifra de bloco		Cifra de fluxo	
Algoritmo	Tamanho da chave	Algoritmo	Tamanho da chave
AES	128, 256	RC4-40	40
IDEA	128	RC4-128	128
RC2-40	40		
DES-40	40		
DES	56		
3DES	168		
Fortezza	80		

Fortezza pode ser usado em um esquema de criptografia de cartão inteligente.

Para a encriptação de fluxo, a mensagem compactada mais o MAC são encriptados. Observe que o MAC é calculado antes que a encriptação ocorra e que o é então encriptado com o texto claro ou texto claro compactado.

Para a encriptação em bloco, o preenchimento pode ser acrescentado após o MAC antes da encriptação. O preenchimento está na forma de um número de bytes de preenchimento seguidos por uma indicação de um byte do tamanho do preenchimento. A quantidade total de preenchimento é a menor, de modo que o tamanho total dos dados a serem encriptados (texto claro mais MAC mais preenchimento) seja um múltiplo do tamanho de bloco da cifra. Um exemplo é um texto claro (ou compactado, se a compactação for usada) de 58 bytes, com um MAC de 20 bytes (usando SHA-1), que é encriptado usando um tamanho de bloco de 8 bytes (por exemplo, DES). Com o byte preenchimento-tamanho, isso gera um total de 79 bytes. Para tornar o total um múltiplo inteiro de 8, um byte de preenchimento é acrescentado.

A última etapa do processamento do protocolo de registro SSL é anexar um cabeçalho no início, consistindo nos seguintes campos:

- **Tipo de conteúdo (8 bits):** o protocolo da camada mais alta, usado para processar o fragmento delimitado.
- **Versão principal (8 bits):** indica a versão principal do SSL em uso. Para SSLv3, o valor é 3.
- **Versão secundária (8 bits):** indica a versão secundária em uso. Para SSLv3, o valor é 0.
- **Tamanho compactado (16 bits):** o tamanho em bytes do fragmento de texto claro (ou fragmento compactado, se a compactação for usada). O valor máximo é $2^{14} + 2048$.

Os tipos de conteúdo que foram definidos são `change_cipher_spec`, `alert`, `handshake` e `application_data`. Os três primeiros são os protocolos específicos do SSL, discutidos a seguir. Observe que nenhuma distinção é feita entre as diversas aplicações (por exemplo, HTTP) que poderiam usar SSL; o conteúdo dos dados criados por tais aplicações é opaco ao SSL. A Figura 17.4 ilustra o formato do registro SSL.

Protocolo de especificação de mudança de cifra

O protocolo de especificação de mudança de cifra é um dos três protocolos específicos do SSL que utilizam o protocolo de registro SSL, e é o mais simples. Ele consiste em uma única mensagem (Figura 17.5a), que consiste em um único byte com o valor 1. A única finalidade dessa mensagem é fazer com que o estado pendente seja copiado para o estado atual, que atualiza o conjunto de cifra a ser usado nessa conexão.

Figura 17.4 Formato do registro SSL.

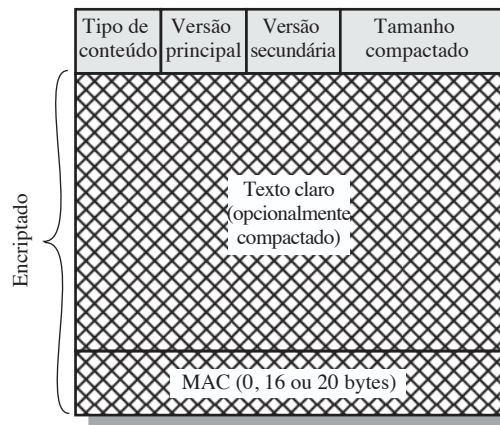
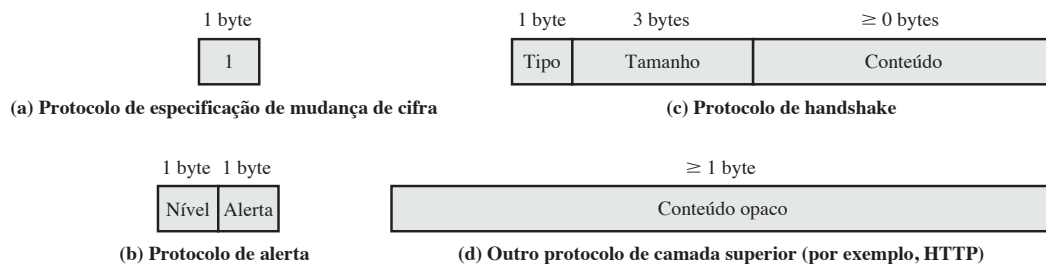


Figura 17.5 Payload do protocolo de registro SSL.



Protocolo de alerta

O protocolo de alerta é usado para transmitir alertas relacionados ao SSL para o par. Assim como outras aplicações que usam SSL, as mensagens de alerta são compactadas e encriptadas, conforme especificado pelo estado atual.

Cada mensagem nesse protocolo consiste em dois bytes (Figura 17.5b). O primeiro pode significar advertência (1) ou fatal (2) para sinalizar a seriedade da mensagem. Se o nível for fatal, o SSL termina imediatamente a conexão. Outras conexões na mesma sessão podem continuar, mas nenhuma conexão nova nessa sessão pode ser estabelecida. O segundo byte contém um código que indica o alerta específico. Primeiro, listamos os alertas que são sempre fatais (definições da especificação SSL).

- **unexpected_message**: uma mensagem não apropriada foi recebida.
- **bad_record_mac**: um MAC incorreto foi recebido.
- **decompression_failure**: a função de descompactação recebeu entrada imprópria (por exemplo, incapaz de descompactar ou descompactar para mais do que o tamanho máximo permitido).
- **handshake_failure**: emissor foi incapaz de negociar um conjunto aceitável de parâmetros de segurança dadas as opções disponíveis.
- **illegal_parameter**: um campo em uma mensagem de handshake estava fora de intervalo ou inconsistente com outros campos.

O restante dos alertas é o seguinte:

- **close_notify**: notifica o destinatário de que o emissor não enviará mais mensagens nessa conexão. Cada parte precisa enviar um alerta **close_notify** antes de fechar o lado de escrita de uma conexão.
- **no_certificate**: pode ser enviado em resposta a uma solicitação de certificado se nenhum certificado apropriado estiver disponível.

- **bad_certificate:** um certificado recebido foi adulterado (por exemplo, continha uma assinatura inválida).
- **unsupported_certificate:** o tipo do certificado recebido não é admitido.
- **certificate_revoked:** um certificado foi revogado por seu assinante.
- **certificate_expired:** um certificado expirou.
- **certificate_unknown:** algum outro problema não especificado surgiu no processamento do certificado, tornando-o inaceitável.

Protocolo de handshake

A parte mais complexa do SSL é o protocolo de handshake. Esse protocolo permite que o servidor e o cliente autenticuem um ao outro e negociem um algoritmo de encriptação e MAC, e chaves criptográficas a serem usadas para proteger dados enviados em um registro SSL. O protocolo de handshake é usado antes que quaisquer dados de aplicação sejam transmitidos.

O protocolo de handshake consiste em uma série de mensagens trocadas por cliente e servidor. Todas estas têm o formato mostrado na Figura 17.5c. Cada mensagem tem três campos:

- **Tipo (1 byte):** indica uma de 10 mensagens. A Tabela 17.2 lista os tipos de mensagem definidos.
- **Tamanho (3 bytes):** o tamanho da mensagem em bytes.
- **Conteúdo (≥ 0 bytes):** os parâmetros associados a essa mensagem; estes são listados na Tabela 17.2.

A Figura 17.6 mostra a troca inicial necessária para estabelecer uma conexão lógica entre cliente e servidor. A troca pode ser vista como tendo quatro fases.

FASE 1. ESTABELECEER CAPACIDADES DE SEGURANÇA

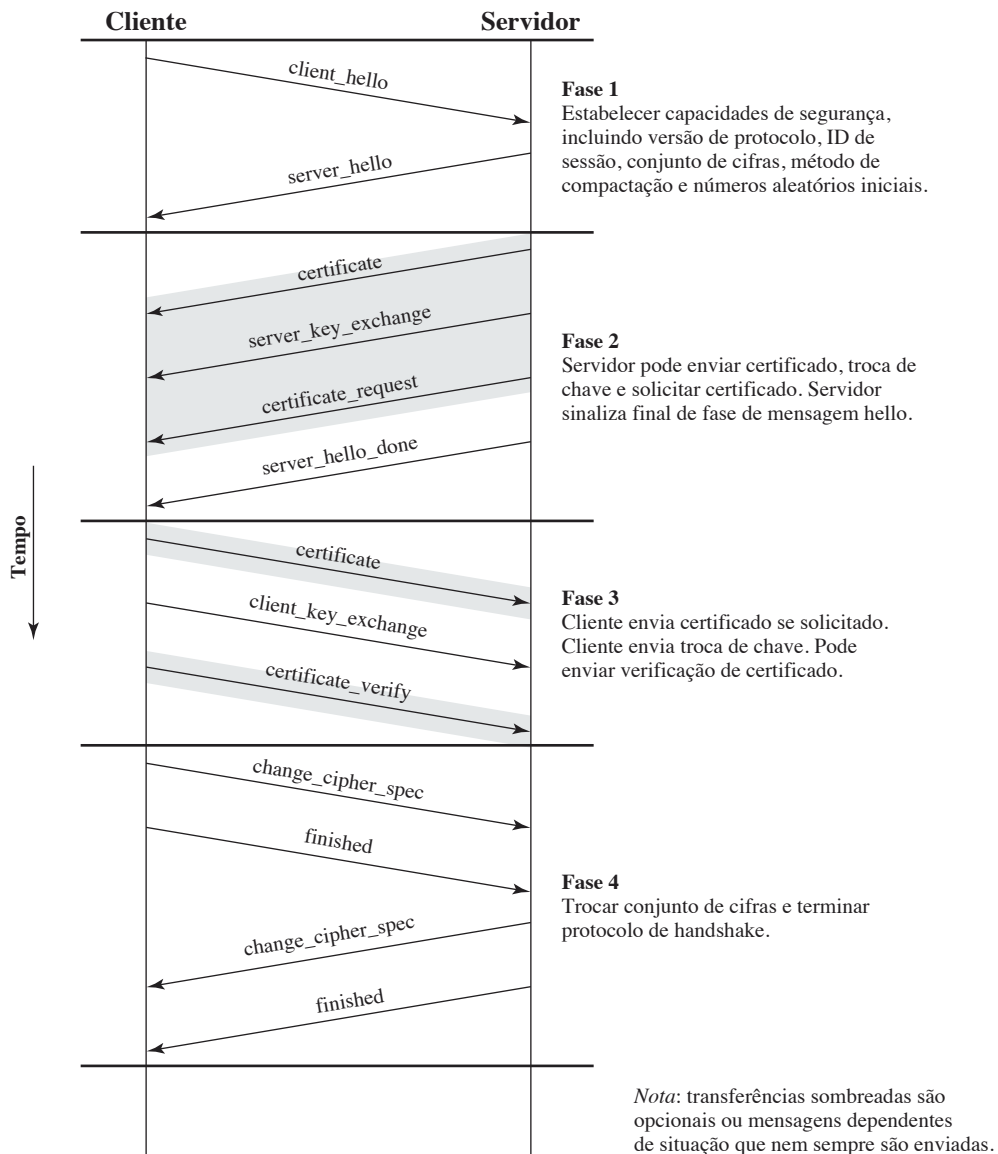
Essa fase é usada para iniciar uma conexão lógica e estabelecer as capacidades de segurança que serão associadas a ela. A troca é iniciada pelo cliente, que envia uma **mensagem client_hello** com os seguintes parâmetros:

- **Versão:** a versão SSL mais alta entendida pelo cliente.
- **Aleatório:** uma estrutura aleatória gerada pelo cliente, consistindo em uma estampa de tempo de 32 bits e 28 bytes gerados por um gerador de número aleatório seguro. Esses valores servem como nonces e são usados durante a troca de chave para impedir ataques de replicação.

Tabela 17.2 Tipos de mensagem do protocolo de handshake SSL.

Tipo de mensagem	Parâmetros
hello_request	nulo
client_hello	versão, aleatório, id de sessão, conjunto de cifras, método de compactação
server_hello	versão, aleatório, id de sessão, conjunto de cifras, método de compactação
certificate	cadeia de certificados X.509v3
server_key_exchange	parâmetros, assinatura
certificate_request	tipo, autoridades
server_done	nulo
certificate_verify	assinatura
client_key_exchange	parâmetros, assinatura
finished	valor de hash

Figura 17.6 Ação do protocolo de handshake.



- **ID de sessão:** um identificador de sessão de tamanho variável. Um valor diferente de zero indica que o cliente deseja atualizar os parâmetros de uma conexão existente ou criar uma nova conexão nesta sessão. Um valor zero indica que o cliente deseja estabelecer uma nova conexão em uma nova sessão.
- **Conjunto de cifras:** essa é uma lista que contém as combinações de algoritmos criptográficos admitidos pelo cliente, em ordem decrescente de preferência. Cada elemento da lista (cada conjunto de cifras) define um algoritmo de troca de chave e uma especificação de cifra; estas serão explicadas posteriormente.
- **Método de compactação:** essa é uma lista dos métodos de compactação que o cliente admite.

Depois de enviar a mensagem `client_hello`, o cliente espera pela mensagem `server_hello`, que contém os mesmos parâmetros da primeira. Para a mensagem `server_hello`, as convenções a seguir se aplicam. O campo Versão contém a menor das versões sugeridas pelo cliente e a maior admitida pelo servidor. O campo Aleatório é gerado pelo servidor e é independente do campo Aleatório do cliente. Se o campo ID de sessão do cliente foi diferente de zero, o mesmo valor é usado pelo servidor; caso contrário, o campo ID de sessão do servidor contém o valor para uma nova sessão. O campo Conjunto de cifras contém o único conjunto de cifras

selecionado pelo servidor a partir daqueles propostos pelo cliente. O campo Compactação contém o método de compactação selecionado pelo servidor a partir daqueles propostos pelo cliente.

O primeiro elemento do parâmetro Conjunto de cifras é o método da troca de chave (ou seja, o meio pelo qual as chaves criptográficas para a encriptação convencional e MAC são trocadas). Os seguintes métodos de troca de chave são admitidos:

- **RSA:** a chave secreta é encriptada com a chave pública RSA do receptor. Um certificado de chave pública para a chave do receptor precisa se tornar disponível.
- **Diffie-Hellman fixo:** essa é uma troca de chave Diffie-Hellman em que o certificado do servidor contém os parâmetros públicos Diffie-Hellman assinados pela autoridade de certificação (CA). Ou seja, o certificado de chave pública contém os parâmetros de chave pública Diffie-Hellman. O cliente oferece esses parâmetros de chave pública Diffie-Hellman ou em um certificado, se a autenticação do cliente for exigida, ou em uma mensagem de troca de chave. Esse método resulta em uma chave secreta fixa entre dois pares, com base no cálculo Diffie-Hellman usando as chaves públicas fixas.
- **Diffie-Hellman efêmero:** essa técnica é usada para criar chaves secretas efêmeras (temporárias, de uso único). Nesse caso, as chaves públicas Diffie-Hellman são trocadas, assinadas usando a chave RSA ou DSS privada do emissor. O receptor pode usar a chave pública correspondente para verificar a assinatura. Os certificados são usados para autenticar as chaves públicas. Isso pareceria ser a mais segura das três opções Diffie-Hellman, pois resulta em uma chave temporária, autenticada.
- **Diffie-Hellman anônimo:** o algoritmo Diffie-Hellman básico é usado sem autenticação. Ou seja, cada lado envia seus parâmetros Diffie-Hellman públicos para o outro sem autenticação. Essa técnica é vulnerável a ataques do tipo *man-in-the-middle*, em que o atacante realiza Diffie-Hellman anônimo com ambas as partes.
- **Fortezza:** a técnica definida para o esquema Fortezza.

Após a definição de um método de troca de chave existe a especificação de cifra, que inclui os seguintes campos:

- **Algoritmo de cifra:** qualquer um dos algoritmos mencionados anteriormente: RC4, RC2, DES, 3DES, DES40, IDEA, ou Fortezza
- **Algoritmo MAC:** MD5 ou SHA-1
- **Tipo de cifra:** fluxo ou bloco
- **É exportável:** verdadeiro ou falso
- **Tamanho de hash:** 0, 16 (para MD5) ou 20 (para SHA-1) bytes
- **Material da chave:** uma sequência de bytes que contém dados usados na geração das chaves de escrita
- **Tamanho do IV:** o tamanho do vetor de inicialização para a encriptação CBC (*Cipher Block Chaining*)

FASE 2. AUTENTICAÇÃO DE SERVIDOR E TROCA DE CHAVE

O servidor inicia essa fase enviando seus certificados, se precisar ser autenticado; a mensagem contém uma ou uma cadeia de certificados X.509. A **mensagem de certificado** é exigida para qualquer método e troca de chave combinado, exceto Diffie-Hellman anônimo. Observe que, se o Diffie-Hellman fixo for usado, essa mensagem de certificado funciona como a mensagem de troca de chave do servidor, pois contém os parâmetros Diffie-Hellman públicos do servidor.

Em seguida, uma **mensagem server_key_exchange** pode ser enviada, se for necessário. Ela não é exigida em dois casos: (1) o servidor enviou um certificado com parâmetros Diffie-Hellman fixos, ou (2) a troca de chave RSA deve ser usada. A mensagem `server_key_exchange` é necessária para o seguinte:

- **Diffie-Hellman anônimo:** o conteúdo da mensagem consiste nos dois valores Diffie-Hellman globais (um número primo e uma raiz primitiva desse número) mais a chave Diffie-Hellman pública do servidor (ver Figura 10.1).

- **Diffie-Hellman efêmero:** o conteúdo da mensagem inclui os três parâmetros Diffie-Hellman fornecidos para Diffie-Hellman anônimo, mais uma assinatura desses parâmetros.
- **Troca de chave RSA (em que o servidor está usando RSA, mas tem uma chave RSA apenas de assinatura):** por conseguinte, o cliente não pode simplesmente enviar uma chave secreta encriptada com a chave pública do servidor. Em vez disso, o servidor precisa criar um par de chaves pública/privada RSA e usar a mensagem `server_key_exchange` para enviar a chave pública. O conteúdo da mensagem inclui os dois parâmetros da chave pública RSA temporária (expoente e módulo; ver Figura 9.5) mais uma assinatura desses parâmetros.
- **Fortezza**

Mais alguns detalhes sobre as assinaturas são garantidos. Como sempre, uma assinatura é criada apanhando-se o hash de uma mensagem e encriptando-o com a chave privada do emissor. Nesse caso, o hash é definido como

```
hash(ClientHello.random || ServerHello.random ||
      ServerParams)
```

Assim, o hash abrange não apenas os parâmetros Diffie-Hellman ou RSA, mas também os dois nonces das mensagens hello iniciais. Isso previne ataques de replicação e erro de representação. No caso de uma assinatura DSS, o hash é realizado usando-se o algoritmo SHA-1. No caso de uma assinatura RSA, tanto MD5 quanto um hash SHA-1 são calculados, e a concatenação dos dois hashes (36 bytes) é encriptada com a chave privada do servidor.

Em seguida, um servidor não anônimo (servidor não usando Diffie-Hellman anônimo) pode solicitar um certificado do cliente. A **mensagem `certificate_request`** inclui dois parâmetros: `certificate_type` e `certificate_authorities`. O tipo de certificado indica o algoritmo de chave pública e seu uso:

- RSA, somente assinatura
- DSS, somente assinatura
- RSA para Diffie-Hellman fixo; nesse caso, a assinatura só é usada para autenticação, enviando um certificado assinado com RSA
- DSS para Diffie-Hellman fixo; novamente, usado apenas para autenticação
- RSA para Diffie-Hellman efêmero
- DSS para Diffie-Hellman efêmero
- Fortezza

O segundo parâmetro na mensagem `certificate_request` é uma lista dos nomes distintos de autoridades certificadoras aceitáveis.

A mensagem final na Fase 2, e que é sempre exigida, é a **mensagem `server_done`**, que é enviada pelo servidor para indicar o final do hello do servidor e mensagens associadas. Depois de enviar essa mensagem, o servidor esperará pela resposta de um cliente. Essa mensagem não possui parâmetros.

FASE 3. AUTENTICAÇÃO DE CLIENTE E TROCA DE CHAVE

Ao receber a mensagem `server_done`, o cliente deve verificar se o servidor forneceu um certificado válido, se exigido, e se os parâmetros `server_hello` são aceitáveis. Se tudo estiver satisfatório, o cliente envia uma ou mais mensagens de volta ao servidor.

Se o servidor tiver solicitado um certificado, o cliente inicia essa fase enviando uma **mensagem de certificado**. Se nenhum certificado apropriado estiver disponível, o cliente envia um alerta `no_certificate` em vez disso.

Em seguida vem a **mensagem `client_key_exchange`**, que precisa ser enviada nessa fase. O conteúdo da mensagem depende do tipo de troca de chave, da seguinte forma:

- **RSA:** o cliente gera um *segredo pré-mestre* de 48 bytes e o encripta com a chave pública do certificado do servidor ou chave RSA temporária de uma mensagem `server_key_exchange`. Seu uso para calcular um *segredo mestre* é explicado mais adiante.
- **Diffie-Hellman efêmero ou anônimo:** os parâmetros Diffie-Hellman públicos do cliente são enviados.
- **Diffie-Hellman fixo:** os parâmetros Diffie-Hellman públicos do cliente foram enviados em uma mensagem de certificado, de modo que o conteúdo dessa mensagem é nulo.
- **Fortezza:** os parâmetros Fortezza do cliente são enviados.

Finalmente, nesta fase, o cliente pode enviar uma **mensagem `certificate_verify`** para oferecer verificação explícita de um certificado do cliente. Essa mensagem só é enviada caso o certificado do cliente tenha capacidade de assinatura (ou seja, todos os certificados exceto aqueles contendo parâmetros Diffie-Hellman fixos). Essa mensagem assina um código de hash com base nas mensagens anteriores, definido da seguinte forma:

```
CertificateVerify.signature.md5_hash =
    MD5(master_secret || pad_2 || MD5(handshake_messages ||
        master_secret || pad_1));
CertificateVerify.signature.sha_hash =
    SHA(master_secret || pad_2 || SHA(handshake_messages ||
        master_secret || pad_1));
```

onde `pad_1` e `pad_2` são os valores definidos anteriormente para o MAC, **`handshake_messages`** refere-se a todas as mensagens Handshake Protocol enviadas ou recebidas desde a mensagem `client_hello`, mas não incluindo essa mensagem, e `master_secret` é o segredo calculado cuja construção é explicada mais adiante nesta seção. Se a chave privada do usuário for DSS, então ela é usada para encriptar o hash SHA-1. Se a chave privada do usuário for RSA, ela é usada para encriptar a concatenação dos hashes MD5 e SHA-1. De qualquer forma, a finalidade é verificar se o cliente possui a chave privada para o certificado do cliente. Mesmo que alguém esteja usando o certificado do cliente sem autorização, ele não conseguiria enviar essa mensagem.

FASE 4. TÉRMINO

Essa fase completa a configuração de uma conexão segura. O cliente envia uma **mensagem `change_cipher_spec`** e copia a especificação de cifra pendente para a especificação de cifra atual. Observe que essa mensagem não é considerada parte do protocolo de handshake, mas é enviada usando o protocolo de especificação de mudança. O cliente, então, envia imediatamente a **mensagem de concluído** sob os novos algoritmos, chaves e segredos. A mensagem de concluído verifica se os processos de troca de chave e autenticação foram bem-sucedidos. O conteúdo da mensagem de concluído é a concatenação de dois valores de hash:

```
MD5(master_secret || pad2 || MD5(handshake_messages ||
    Sender || master_secret || pad1))
SHA(master_secret || pad2 || SHA(handshake_messages ||
    Sender || master_secret || pad1))
```

onde `Sender` é um código que identifica que o emissor é o cliente e `handshake_messages` são todos os dados de todas as mensagens de handshake até então, mas não incluindo esta mensagem.

Em resposta a essas duas mensagens, o servidor envia sua própria mensagem `change_cipher_spec`, transfere a especificação de cifra pendente para a atual e envia sua mensagem de concluído. Nesse ponto, o handshake está completo e cliente e servidor podem começar a trocar dados da camada de aplicação.

Cálculos criptográficos

Dois outros itens são interessantes: (1) a criação de uma chave mestra compartilhada por meio da troca de chave e (2) a geração de parâmetros criptográficos do segredo mestre.

criação de segredo mestre

O segredo mestre compartilhado é um valor de 48 bytes (384 bits) de uso único gerado para esta sessão por meio da troca de chave segura. A criação é feita em dois estágios. Primeiro, um `pre_master_secret` é trocado. Segundo, o `master_secret` é calculado pelas duas partes. Para a troca do `pre_master_secret`, existem duas possibilidades:

- **RSA:** um `pre_master_secret` de 48 bytes é gerado pelo cliente, encriptado com a chave RSA pública do servidor e enviado ao servidor. Ele decifra o texto cifrado usando sua chave privada para recuperar o `pre_master_secret`.
- **Diffie-Hellman:** cliente e servidor geram uma chave pública Diffie-Hellman. Depois que estes forem trocados, cada lado realiza o cálculo Diffie-Hellman para criar o `pre_master_secret` compartilhado.

Os dois lados agora calculam o `master_secret` da seguinte forma:

```
master_secret = MD5(pre_master_secret || SHA('A' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random)) ||
MD5(pre_master_secret || SHA('BB' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random)) ||
MD5(pre_master_secret || SHA('CCC' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random))
```

onde `ClientHello.random` e `ServerHello.random` são os dois valores de nonce trocados nas mensagens hello iniciais.

geração de parâmetros criptográficos

CipherSpecs requer um segredo MAC de escrita do cliente, um segredo MAC de escrita do servidor, uma chave de escrita do cliente, uma chave de escrita do servidor, um IV de escrita do cliente e um IV de escrita do servidor, que são gerados a partir do segredo mestre nessa ordem. Esses parâmetros são gerados a partir do segredo mestre pelo hashing do segredo mestre para uma sequência de bytes seguros de tamanho suficiente para todos os parâmetros necessários.

A geração do material de chave a partir do `master_secret` usa o mesmo formato para a geração do segredo mestre a partir do `pre_master_secret`:

```
key_block = MD5(master_secret || SHA('A' || master_secret ||
                    ServerHello.random || ClientHello.random)) ||
MD5(master_secret || SHA('BB' || master_secret ||
                    ServerHello.random || ClientHello.random)) ||
MD5(master_secret || SHA('CCC' || master_secret ||
                    ServerHello.random || ClientHello.random)) ||...
```

até que uma saída suficiente tenha sido gerada. O resultado dessa estrutura algorítmica é uma função pseudoaleatória. Podemos ver `master_secret` como o valor de semente pseudoaleatória para a função. Os números aleatórios de cliente e servidor podem ser vistos como valores de sal para complicar a criptoanálise.

17.3 TRANSPORT LAYER SECURITY

TLS é uma iniciativa de padronização do IETF cujo objetivo é produzir uma versão padrão do SSL para Internet. TLS é definido como um Proposed Internet Standard na RFC 5246. A RFC 5246 é muito semelhante à SSLv3. Nesta seção, destacamos as diferenças.

Número de versão

O formato de registro do TLS é o mesmo do SSL (Figura 174), e os campos no cabeçalho têm os mesmos significados. A única diferença está nos valores de versão. Para a versão atual do TLS, a principal é 3 e a secundária é 3.

Message Authentication Code

Existem duas diferenças entre os esquemas de MAC do SSLv3 e do TLS: o algoritmo real e o escopo do cálculo do MAC. O TLS utiliza o algoritmo HMAC definido na RFC 2104. Lembre-se, do Capítulo 12, que o HMAC é definido da seguinte forma:

$$\text{HMAC}_K(M) = H[(K^+ \oplus \text{opad}) \parallel H[K^+ \oplus \text{ipad}] \parallel M]$$

onde

- H = função de hash embutida (para o TLS, pode ser MD5 ou SHA-1)
- M = entrada de mensagem para HMAC
- K^+ = chave secreta preenchida com zeros à esquerda, de modo que o resultado é igual ao tamanho do bloco do código de hash (para MD5 e SHA-1, tamanho de bloco = 512 bits)
- ipad = 00110110 (36 em hexadecimal) repetido 64 vezes (512 bits)
- opad = 01011100 (5C em hexadecimal) repetido 64 vezes (512 bits)

SSLv3 usa o mesmo algoritmo, exceto que os bytes de preenchimento são concatenados com a chave secreta em vez de passarem por um XOR com a chave secreta preenchida até o tamanho do bloco. O nível de segurança deverá ser praticamente o mesmo nos dois casos.

Para o TLS, o cálculo do MAC compreende os campos indicados na expressão a seguir:

```
MAC(MAC_write_secret, seq_num || TLSCompressed.type ||
    TLSCompressed.version || TLSCompressed.length ||
    TLSCompressed.fragment)
```

O cálculo do MAC abrange todos os campos cobertos pelo cálculo do SSLv3, mais o campo `TLSCompressed.version`, que é a versão do protocolo sendo empregada.

Função pseudoaleatória

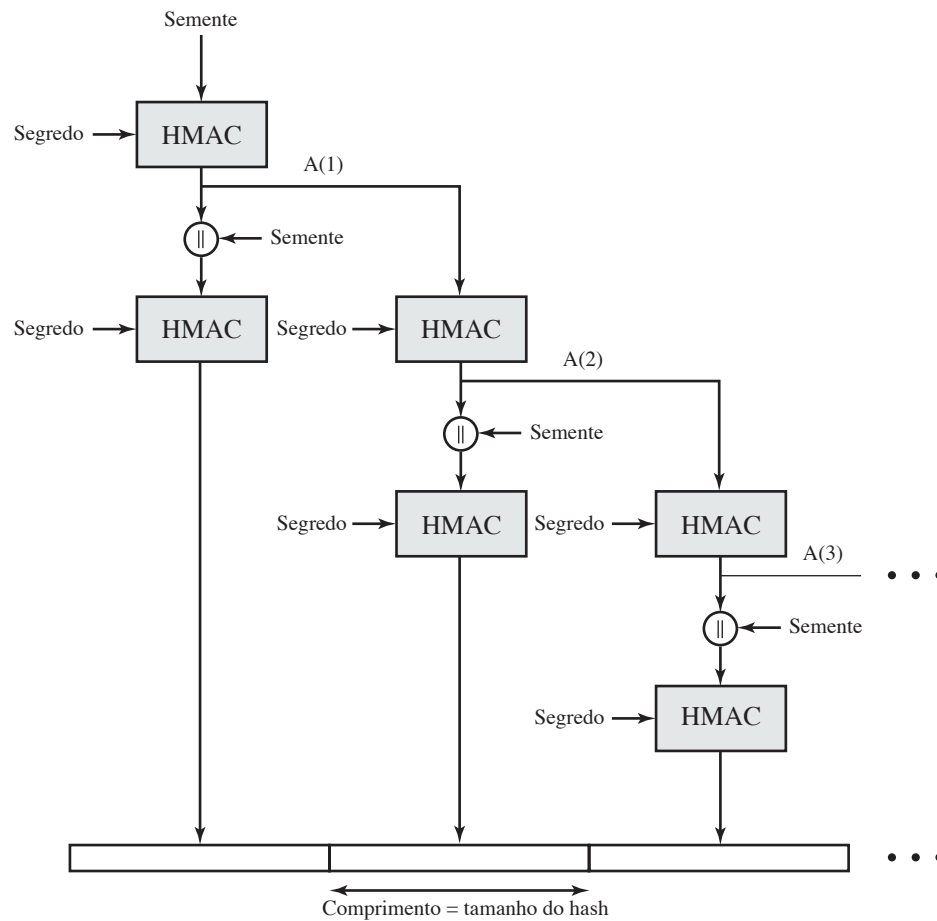
O TLS utiliza uma função pseudoaleatória referenciada como PRF (acrônimo em inglês para *Pseudorandom Function*) para expandir segredos em blocos de dados para fins de geração ou validação de chave. O objetivo é utilizar um valor de segredo compartilhado relativamente pequeno, mas gerar blocos de dados maiores de um modo que seja seguro contra os tipos de ataques feitos sobre funções de hash e MACs. O PRF é baseado na seguinte função de expansão de dados (Figura 177):

```
P_hash(segredo, semente) = HMAC_hash(segredo, A(1) || semente) ||
                           HMAC_hash(segredo, A(2) || semente) ||
                           HMAC_hash(segredo, A(3) || semente) || ...
```

onde $A()$ é definido como

```
A(0) = semente
A(i) = HMAC_hash(segredo, A(i-1))
```

A função de expansão de dados utiliza o algoritmo HMAC, seja com MD5 ou SHA-1, como função de hash básica. Como podemos ver, `P_hash` pode ser repetido tantas vezes quantas forem necessárias para produzir a quantidade de dados exigida. Por exemplo, se `P_SHA-1` fosse usado para gerar 64 bytes de dados, ele teria que ser repetido quatro vezes, produzindo 80 bytes de dados, dos quais os últimos 16 seriam descartados. Nesse caso, `P_MD5` também teria que ser repetido quatro vezes, produzindo exatamente 64 bytes de dados. Observe

Figura 17.7 Função $P_hash(segredo, semente)$ do TLS.

que cada repetição envolve duas execuções do HMAC, cada uma por sua vez envolvendo duas execuções do algoritmo de hash básico.

Para tornar o PRF tão seguro quanto possível, ele usa dois algoritmos de hash de um modo que garanta sua segurança se um deles permanecer seguro. PRF é definido como

$$\text{PRF}(\text{segredo}, \text{rótulo}, \text{semente}) = P_hash(S1, \text{rótulo} || \text{semente})$$

PRF recebe como entrada um valor secreto, um rótulo de identificação e um valor de semente e produz uma saída de tamanho arbitrário.

Códigos de alerta

TLS admite todos os códigos de alerta definidos na SSLv3, com a exceção de `no_certificate`. Diversos códigos adicionais são definidos no TLS; destes, os seguintes são sempre fatais:

- **record_overflow**: um registro do TLS foi recebido com um payload (texto cifrado) cujo tamanho ultrapassa $2^{14} + 2048$ bytes, ou o texto cifrado decifrado para um tamanho maior que $2^{14} + 1024$ bytes.
- **unknown_ca**: uma cadeia de certificado válida ou parcial foi recebida, mas o certificado não foi aceito porque o certificado da CA não pôde ser localizado ou não pôde ser combinado com uma CA conhecida e confiável.
- **access_denied**: um certificado válido foi recebido, mas quando a conectividade de acesso foi aplicada, o emissor decidiu não prosseguir com a negociação.
- **decode_error**: uma mensagem não pôde ser decodificada, pois um campo estava fora do seu intervalo especificado ou o tamanho da mensagem foi incorreto.

- **protocol_version**: a versão do protocolo que o cliente tentou negociar é reconhecida, mas não é aceita.
- **insufficient_security**: retornado em vez de `handshake_failure` quando uma negociação falhou especificamente porque o servidor exige cifras mais seguras do que aquelas admitidas pelo cliente.
- **unsupported_extension**: enviado pelos clientes que recebem um hello do servidor estendido contendo uma extensão não incluída no hello do cliente correspondente.
- **internal_error**: um erro interno não relacionado ao par ou à exatidão do protocolo torna impossível continuar.
- **decrypt_error**: uma operação criptográfica de handshake falhou, incluindo ser incapaz de verificar uma assinatura, decryptar uma troca de chave ou validar uma mensagem acabada.

O restante dos alertas inclui o seguinte:

- **user_canceled**: esse handshake está sendo cancelado por algum motivo não relacionado a uma falha de protocolo.
- **no_renegotiation**: enviado por um cliente em resposta a uma solicitação hello ou pelo servidor em resposta a um hello do cliente após o handshaking inicial. Uma dessas mensagens normalmente resultaria em renegociação, mas esse alerta indica que o emissor não é capaz de renegociar. Essa mensagem é sempre uma advertência.

Conjuntos de cifras

Existem várias diferenças pequenas entre os conjuntos de cifras disponíveis sob SSLv3 e sob TLS:

- **Troca de chave**: o TLS admite todas as técnicas de troca de chave do SSLv3, com a exceção do Fortezza.
- **Algoritmos de criptografia simétrica**: o TLS inclui todos os algoritmos de criptografia simétrica encontrados no SSLv3, com a exceção do Fortezza.

Tipos de certificado do cliente

O TLS define os seguintes tipos de certificado a serem solicitados em uma mensagem `certificate_request`: `rsa_sign`, `dss_sign`, `rsa_fixed_dh` e `dss_fixed_dh`. Todos esses são definidos no SSLv3. Além disso, SSLv3 inclui `rsa_ephemeral_dh`, `dss_ephemeral_dh` e `fortezza_kea`. O Diffie-Hellman efêmero envolve a assinatura dos parâmetros Diffie-Hellman com RSA ou DSS. Para o TLS, os tipos `rsa_sign` e `dss_sign` são usados para essa função; um tipo de assinatura separado não é necessário para assinar parâmetros Diffie-Hellman. O TLS não inclui o esquema Fortezza.

Certificate_verify e mensagens de concluído

Na mensagem `certificate_verify`, os hashes MD5 e SHA-1 são calculados apenas sobre `handshake_messages`. Lembre-se de que, para o SSLv3, o cálculo de hash também incluía o segredo mestre e preenchimentos. Esses campos extras não acrescentam segurança adicional.

Assim como a mensagem de concluído no SSLv3, a mensagem de concluído no TLS é um hash com base no `master_secret` compartilhado, as mensagens de handshake anteriores, e um rótulo que identifica cliente ou servidor. O cálculo é um pouco diferente. Para o TLS, temos

$$\text{PRF}(\text{master_secret}, \text{finished_label}, \text{MD5}(\text{handshake_messages}) || \text{SHA-1}(\text{handshake_messages}))$$

onde `finished_label` é a string “client finished” para o cliente e “server finished” para o servidor.

Cálculos criptográficos

O `pre_master_secret` para o TLS é calculado da mesma maneira que no SSLv3. Assim como no SSLv3, o `master_secret` no TLS é calculado como uma função de hash do `pre_master_secret` e os dois números aleatórios de hello. A forma do cálculo do TLS é diferente do que é usado no SSLv3, e é definida da seguinte forma:

```
master_secret = PRF(pre_master_secret, "master secret",
                    ClientHello.random || ServerHello.random)
```

O algoritmo é executado até que 48 bytes de saída pseudoaleatória sejam produzidos. O cálculo do material do bloco de chave (chaves secretas MAC, chaves de encriptação de sessão e IVs) é definido da seguinte forma:

```
key_block = PRF(master_secret, "key expansion",
                SecurityParameters.server_random ||
                SecurityParameters.client_random)
```

até que uma saída suficiente tenha sido gerada. Assim como no SSLv3, `key_block` é uma função da `master_secret` e os números aleatórios de cliente e servidor, mas, para o TLS, o algoritmo real é diferente.

Preenchimento

No SSL, o preenchimento acrescentado antes da encriptação de dados do usuário é a quantidade mínima exigida para que o tamanho total dos dados a serem encriptados seja um múltiplo do tamanho de bloco da cifra. No TLS, o preenchimento pode ser qualquer quantidade que resulte em um total que seja um múltiplo do tamanho de bloco da cifra, até um máximo de 255 bytes. Por exemplo, se o texto claro (ou texto compactado, se a compactação for usada) mais MAC mais byte do tamanho do preenchimento tiver 79 bytes de extensão, então o tamanho do preenchimento, em bytes, pode ser 1, 9, 17 e assim por diante, até 249. Um tamanho de preenchimento variável pode ser usado para frustrar ataques com base em uma análise dos tamanhos das mensagens trocadas.

17.4 HTTPS

HTTPS (HTTP over SSL) refere-se à combinação de HTTP e SSL para implementar a comunicação segura entre um navegador Web e um servidor Web. A capacidade do HTTPS está embutida em todos os navegadores Web modernos. Seu uso depende do servidor Web que dá suporte à comunicação HTTPS. Por exemplo, alguns mecanismos de busca não admitem HTTPS. O Google oferece HTTPS como uma opção: <https://google.com>.

A diferença principal vista por um usuário de um navegador Web é que os endereços de URL (acrônimo em inglês para *Uniform Resource Locator*) começam com `https://` em vez de `http://`. Uma conexão HTTP normal usa a porta 80. Se for especificado HTTPS, a porta 443 é usada, que invoca o SSL.

Quando o HTTPS é usado, os seguintes elementos da comunicação são encriptados:

- URL do documento solicitado
- Conteúdo do documento
- Conteúdo dos formulários do navegador (preenchidos pelo usuário do navegador)
- Cookies enviados do navegador ao servidor e do servidor ao navegador
- Conteúdo do cabeçalho HTTP

HTTPS é documentado na RFC 2818, *HTTP Over TLS*. Não existe mudança fundamental no uso de HTTP sobre SSL ou TLS, e as duas implementações são conhecidas como HTTPS.

Início de conexão

Para HTTPS, o agente atuando como cliente HTTP também atua como cliente TLS. O cliente inicia uma conexão com o servidor na porta apropriada e depois envia o `ClientHello` do TLS para iniciar o handshake TLS. Quando o handshake TLS tiver terminado, o cliente pode então iniciar a primeira solicitação HTTP. Todos os dados HTTP devem ser enviados como dados de aplicação TLS. O comportamento normal do HTTP, incluindo conexões retidas, deve ser seguido.

Há três níveis de percepção de uma conexão no HTTPS. No nível HTTP, um cliente HTTP solicita uma conexão com um servidor HTTP enviando uma solicitação de conexão à próxima camada de nível mais baixo. Normalmente, a próxima camada mais baixa é o TCP, mas também pode ser TLS/SSL. No nível do TLS, uma sessão é estabelecida entre um cliente TLS e um servidor TLS. Essa sessão pode admitir uma ou mais conexões em determinado momento. Como vimos, uma solicitação de TLS para estabelecer uma conexão começa com o estabelecimento de uma conexão TCP entre a entidade TCP no lado do cliente e a entidade TCP no lado do servidor.

Fechamento de conexão

Um cliente ou servidor HTTP pode indicar o fechamento de uma conexão incluindo a seguinte linha em um registro HTTP: `Connection: close`. Isso indica que a conexão será fechada após esse registro ser entregue.

O fechamento de uma conexão HTTPS requer que o TLS feche a conexão com a entidade TLS correspondente no lado remoto, o que envolverá o fechamento da conexão TCP subjacente. No nível do TLS, o modo correto de fechar uma conexão é que cada lado use o protocolo de alerta do TLS para enviar um alerta `close_notify`. As implementações do TLS deverão iniciar uma troca de alertas de fechamento antes que a conexão seja fechada. Uma implementação de TLS pode, depois de enviar um alerta de fechamento, gerar um “fechamento incompleto”. Observe que uma implementação que faz isso pode decidir reutilizar a sessão. Isso só deverá ser feito quando as aplicações souberem (normalmente, através da detecção dos limites da mensagem HTTP) que receberam todos os dados de mensagem que lhes interessam.

Clientes HTTP também deverão ser capazes de lidar com uma situação em que a conexão TCP subjacente é terminada sem um alerta `close_notify` anterior e sem um indicador `Connection: close`. Essa situação poderia ser decorrente de um erro de programação no servidor ou um erro de comunicação que faz com que a conexão TCP caia. Porém, o fechamento não anunciado do TCP poderia ser evidência de algum tipo de ataque. Assim, o cliente HTTPS deverá emitir algum tipo de aviso de segurança quando isso ocorrer.

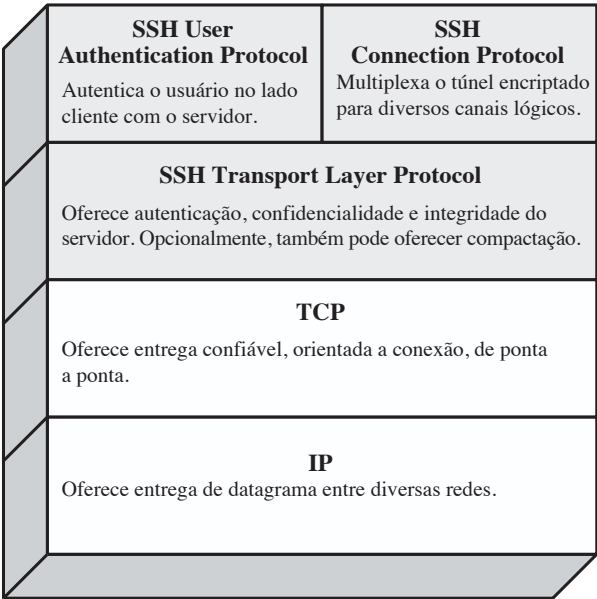
17.5 SECURE SHELL (SSH)

Secure Shell (SSH) é um protocolo para as comunicações de rede seguras, projetado para ser relativamente simples e pouco dispendioso de ser implementado. A versão inicial, SSH1, focalizou o fornecimento de uma facilidade de logon remoto seguro para substituir TELNET e outros esquemas de logon remoto, que não ofereciam segurança. SSH também oferece uma capacidade cliente/servidor mais genérica, e pode ser usado para funções de rede como transferência de arquivos e e-mail. Uma nova versão, SSH2, resolve uma série de falhas de segurança existentes no esquema original. SSH2 é documentado como um padrão proposto nas RFCs 4250 a 4256 do IETF.

Aplicações SSH cliente e servidor podem ser facilmente encontradas para a maioria dos sistemas operacionais. SSH tornou-se o método preferido para login remoto e tunelamento X, e está rapidamente se tornando uma das aplicações mais difundidas para a tecnologia de encriptação fora dos sistemas embutidos.

SSH é organizado como três protocolos, que normalmente são executados em cima do TCP (Figura 17.8):

Figura 17.8 Pilha de protocolos SSH.



- **Transport Layer Protocol:** oferece autenticação de servidor, confidencialidade de dados e integridade de dados com sigilo *forward secrecy* (ou seja se uma chave for comprometida durante uma sessão, o conhecimento não afeta a segurança das seções anteriores). Opcionalmente, a camada de transporte pode oferecer compactação.
- **User Authentication Protocol:** autentica o usuário com o servidor.
- **Connection Protocol:** multiplexa diversos canais de comunicação lógicos sobre uma única conexão SSH subjacente.

Protocolo da Camada de Transporte

CHAVES DE HOSPEDEIRO

A autenticação do servidor ocorre na camada de transporte, com base no servidor processando um par de chaves pública/privada. Um servidor pode ter diversas chaves de hospedeiro usando diversos algoritmos de encriptação assimétrica diferentes. Diversos hospedeiros podem compartilhar a mesma chave. De qualquer forma, a chave de hospedeiro do servidor é usada durante a troca de chaves para autenticar a identidade do hospedeiro. Para que isso seja possível, o cliente precisa ter um conhecimento prévio da chave de hospedeiro pública do servidor. A RFC 4251 indica dois modelos de confiança alternativos que podem ser usados:

1. O cliente tem um banco de dados local que associa cada nome de hospedeiro (conforme digitado pelo usuário) com a chave de hospedeiro pública correspondente. Esse método não requer uma infraestrutura administrada de forma central e nem a coordenação com terceiros. A desvantagem é que a manutenção dos bancos de dados de associações nome-chave pode se tornar trabalhosa.
2. A associação entre nome de hospedeiro e chave é certificada por uma autoridade de certificação (CA) confiável. O cliente só conhece a chave raiz da CA e pode verificar a validade de todas as chaves de hospedeiro certificadas pelas CAs aceitas. Essa alternativa resolve o problema da manutenção porque, de forma ideal, somente uma única chave da CA precisa ser armazenada com segurança no cliente. Por outro lado, cada chave de hospedeiro precisa ser certificada de modo apropriado por uma autoridade central antes que a autorização seja possível.

TROCA DE PACOTES

A Figura 17.9 ilustra a sequência de eventos no Transport Layer Protocol do SSH. Primeiro, o cliente estabelece uma conexão TCP com o servidor. Isso é feito por meio do protocolo TCP e não faz parte do Transport Layer Protocol. Quando a conexão é estabelecida, cliente e servidor trocam dados, conhecidos como pacotes, no campo de dados de um segmento TCP. Cada pacote está no formato a seguir (Figura 17.10).

- **Tamanho do pacote:** comprimento do pacote em bytes, sem incluir os campos de tamanho do pacote e MAC.
- **Tamanho do preenchimento:** comprimento do campo de preenchimento aleatório.
- **Payload:** carga útil do pacote. Antes da negociação do algoritmo, este campo é descompactado. Se a compactação for negociada, então, nos pacotes subsequentes, este campo é compactado.
- **Preenchimento aleatório:** quando um algoritmo de encriptação tiver sido negociado, este campo é acrescentado. Ele contém bytes aleatórios de preenchimento, de modo que o tamanho total do pacote (excluindo o campo MAC) é um múltiplo do tamanho do bloco de cifra, ou 8 bytes para uma cifra de fluxo.
- **Message Authentication Code (MAC):** se a autenticação da mensagem tiver sido negociada, este campo contém um valor MAC. O valor MAC é calculado sobre o pacote inteiro mais um número de sequência, excluindo o campo MAC. O número de sequência é uma sequência de pacotes implícita de 32 bits, que é zerada para o primeiro pacote e incrementada para cada um. O número de sequência não está incluído no pacote enviado pela conexão TCP.

Quando um algoritmo de encriptação tiver sido negociado, o pacote inteiro (excluindo o campo MAC) é encriptado após o valor MAC ser calculado.

Figura 17.9 Trocas de Pacote no SSH Transport Layer Protocol.

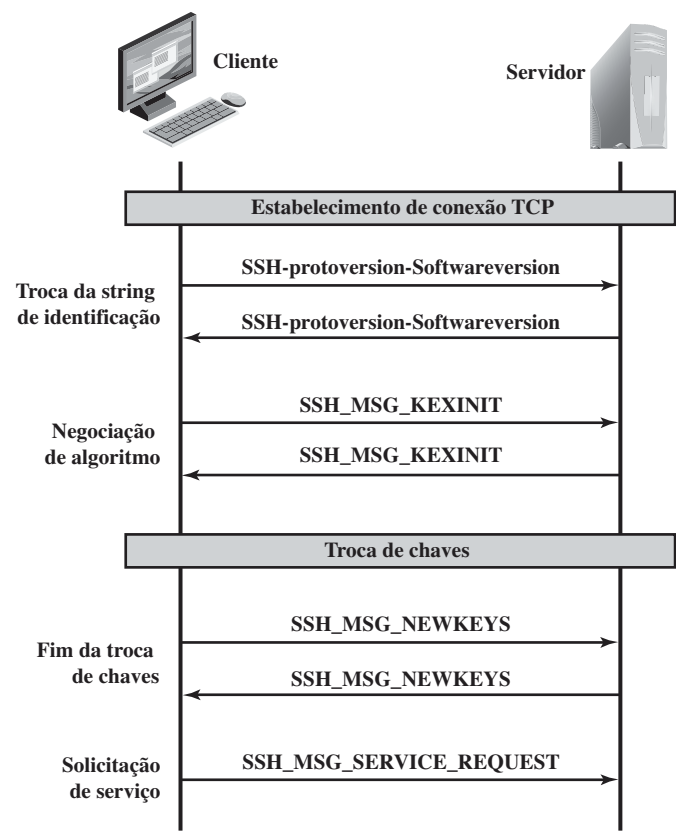
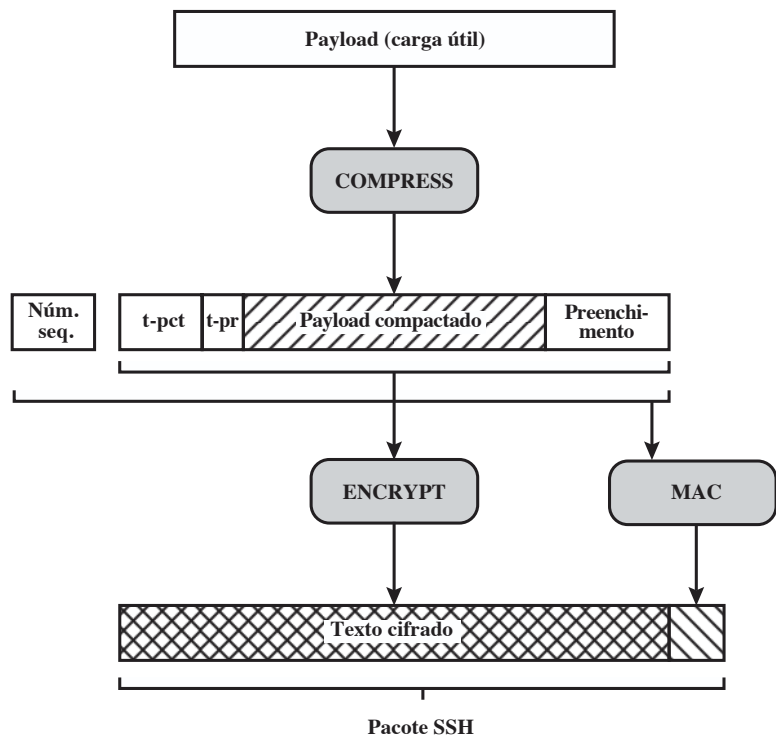


Figura 17.10 Formação do Pacote no SSH Transport Layer Protocol.



t-pct = tamanho do pacote
t-pr = tamanho do preenchimento

A troca de pacotes do SSH Transport Layer consiste em uma sequência de etapas (Figura 179). A primeira, a **troca da string de identificação**, começa com o cliente enviando um pacote com uma string de identificação na forma:

SSH-*protoversion*-*softwareversion* *SP* *comments* *CR* *LF*

onde *SP*, *CR* e *LF* são o caractere de espaço, carriage return e line feed, respectivamente. Um exemplo de uma string válida é `SSH-2.0-blissSSH_3.6.3q3<CR><LF>`. O servidor responde com sua própria string de identificação. Essas strings são usadas na troca de chaves Diffie-Hellman.

Em seguida vem a **negociação do algoritmo**. Cada lado envia uma `SSH_MSG_KEXINIT` contendo listas de algoritmos aceitos na ordem de preferência para o emissor. Há uma lista para cada tipo de algoritmo criptográfico. Os algoritmos incluem troca de chave, encriptação, algoritmo MAC e algoritmo de compactação. A Tabela 17.3 mostra as opções permissíveis para encriptação, MAC e compactação. Para cada categoria, o algoritmo escolhido é o primeiro na lista do cliente que também é aceito pelo servidor.

O próximo passo é a **troca de chaves**. A especificação permite o uso de métodos alternativos de troca de chave, mas, no momento, somente duas versões da troca de chaves Diffie-Hellman foram especificadas. As duas versões são definidas na RFC 2409 e exigem apenas um pacote em cada direção. As etapas a seguir estão envolvidas na troca. Nesta, *C* é o cliente; *S* é o servidor; *p* é um número primo grande e seguro; *g* é um gerador para um subgrupo de $GF(p)$; *q* é a ordem do subgrupo; *V_S* é a string de identificação de *S*; *V_C* é a string de identificação de *C*; *K_S* é a chave de hospedeiro pública de *S*; *I_C* é a mensagem `SSH_MSG_KEXINIT` de *C* e *I_S* é a mensagem `SSH_MSG_KEXINIT` de *S* que foi trocada antes do início desta parte. Os valores de *p*, *g* e *q* são conhecidos do cliente e do servidor como resultado da negociação de seleção de algoritmo. A função de hash `hash()` também é decidida durante a negociação do algoritmo.

1. *C* gera um número aleatório x ($1 < x < q$) e calcula $e = g^x \bmod p$. *C* envia *e* para *S*.

Tabela 17.3 Algoritmos criptográficos do SSH Transport Layer.

Cifra	
3des-cbc*	3DES com três chaves no modo CBC
blowfish-cbc	Blowfish no modo CBC
twofish256-cbc	Twofish no modo CBC com chave de 256 bits
twofish192-cbc	Twofish com chave de 192 bits
twofish128-cbc	Twofish com chave de 128 bits
aes256-cbc	AES no modo CBC com chave de 256 bits
aes192-cbc	AES com chave de 192 bits
aes128-cbc**	AES com chave de 128 bits
Serpent256-cbc	Serpent no modo CBC com chave de 256 bits
Serpent192-cbc	Serpent com chave de 192 bits
Serpent128-cbc	Serpent com chave de 128 bits
arcfour	RC4 com chave de 128 bits
cast128-cbc	CAST-128 no modo CBC

Algoritmo MAC	
hmac-sha1*	HMAC-SHA1; tamanho do resumo = tamanho da chave = 20
hmac-sha1-96**	Primeiros 96 bits do HMAC-SHA1; tamanho do resumo = 12; tamanho da chave = 20
hmac-md5	HMAC-MD5; tamanho do resumo = tamanho da chave = 16
hmac-md5-96	Primeiros 96 bits do HMAC-MD5; tamanho do resumo = 12; tamanho da chave = 16

Algoritmo de compactação	
none*	Sem compactação
zlib	Definido na RFC 1950 e RFC 1951

* = Exigido

** = Recomendado

2. S gera um número aleatório y ($0 < y < q$) e calcula $f = g^y \bmod p$. S recebe e . Ele calcula $K = e^y \bmod p$, $H = \text{hash}(V_C \parallel V_S \parallel I_C \parallel I_S \parallel K_S \parallel e \parallel f \parallel K)$, e assinatura s em H com sua chave de hospedeiro privada. S envia $(K_S \parallel f \parallel s)$ para C. A operação de assinatura pode envolver uma segunda operação de hashing.
3. C verifica se K_S realmente é a chave de hospedeiro para S (por exemplo, usando certificados ou um banco de dados local). C também tem permissão para aceitar a chave sem verificação; porém, isso tornará o protocolo inseguro contra ataques ativos (mas pode ser desejável por motivos práticos a curto prazo em muitos ambientes). C, então, calcula $K = f^x \bmod p$, $H = \text{hash}(V_C \parallel V_S \parallel I_C \parallel I_S \parallel K_S \parallel e \parallel f \parallel K)$, e verifica a assinatura s em H .

Como resultado dessas etapas, os dois lados agora compartilham uma chave secreta K . Além disso, o servidor foi autenticado com o cliente, pois o servidor usou sua chave privada para assinar sua metade da troca Diffie-Hellman. Por fim, o valor de hash H serve como um identificador de sessão para esta conexão. Uma vez calculado, o identificador de sessão não é alterado, mesmo que a troca de chave seja realizada novamente para esta conexão, a fim de obter novas chaves.

O **fim da troca de chaves** é sinalizado pela troca de pacotes `SSH_MSG_NEWKEYS`. Neste ponto, os dois lados podem começar a usar as chaves geradas a partir de K , conforme discutiremos mais adiante.

A etapa final é **solicitação de serviço**. O cliente envia um pacote `SSH_MSG_SERVICE_REQUEST` para solicitar ou o User Authentication Protocol ou o Connection Protocol. Depois disso, todos os dados são trocados como o payload de um pacote SSH Transport Layer, protegido por encriptação e MAC.

GERAÇÃO DE CHAVE

As chaves usadas para encriptação e MAC (e quaisquer IVs necessários) são geradas a partir da chave secreta compartilhada K , do valor de hash da troca de chaves H e do identificador de sessão, que é igual a H , a menos que haja uma troca de chaves subsequente após a troca inicial. Os valores são calculados da seguinte forma:

- IV inicial cliente para servidor: $\text{HASH}(K \parallel H \parallel \text{"A"} \parallel \text{session_id})$
- IV inicial servidor para cliente: $\text{HASH}(K \parallel H \parallel \text{"B"} \parallel \text{session_id})$
- Chave de encriptação cliente para servidor: $\text{HASH}(K \parallel H \parallel \text{"C"} \parallel \text{session_id})$
- Chave de encriptação servidor para cliente: $\text{HASH}(K \parallel H \parallel \text{"D"} \parallel \text{session_id})$
- Chave de integridade cliente para servidor: $\text{HASH}(K \parallel H \parallel \text{"E"} \parallel \text{session_id})$
- Chave de integridade servidor para cliente: $\text{HASH}(K \parallel H \parallel \text{"F"} \parallel \text{session_id})$

onde $\text{HASH}()$ é a função de hash determinada durante a negociação do algoritmo.

Protocolo de Autenticação de Usuário

O User Authentication Protocol oferece meios pelos quais o cliente é autenticado com o servidor.

TIPOS E FORMATOS DE MENSAGEM

Três tipos de mensagens sempre são usadas no User Authentication Protocol. Solicitações de autenticação do cliente têm o formato:

byte	<code>SSH_MSG_USERAUTH_REQUEST</code> (50)
string	nome do usuário
string	nome do serviço
string	nome do método
...	campos específicos do método

onde “nome do usuário” é a identidade de autorização que o cliente está alegando, “nome do serviço” é o serviço ao qual o cliente está solicitando acesso (normalmente, o SSH Connection Protocol) e “nome do método” é o método de autenticação sendo usado nesta solicitação. O primeiro byte tem valor decimal 50, que é interpretado como `SSH_MSG_USERAUTH_REQUEST`.

Se o servidor (1) rejeitar a solicitação de autenticação ou (2) aceitar a solicitação mas exigir um ou mais métodos de autenticação adicionais, o servidor enviará uma mensagem com o formato:

```
byte      SSH_MSG_USERAUTH_FAILURE (51)
name-list autenticações que podem continuar
boolean   sucesso parcial
```

onde a “name-list” é uma lista de métodos que podem produtivamente continuar o diálogo. Se o servidor aceitar autenticação, ele enviará uma mensagem de único byte: `SSH_MSG_USERAUTH_SUCCESS` (52).

TROCA DE MENSAGENS

A troca de mensagens envolve as seguintes etapas:

1. O cliente envia um `SSH_MSG_USERAUTH_REQUEST` com um método solicitado `none`.
2. O servidor verifica e determina se o nome do usuário é válido. Se não, o servidor retorna `SSH_MSG_USERAUTH_FAILURE` com o valor `false` no campo de sucesso parcial. Se o nome do usuário for válido, o servidor prossegue para a etapa 3.
3. O servidor retorna `SSH_MSG_USERAUTH_FAILURE` com uma lista de um ou mais métodos de autenticação a serem usados.
4. O cliente seleciona um dos métodos de autenticação aceitáveis e envia um `SSH_MSG_USERAUTH_REQUEST` com esse nome de método e os campos obrigatórios específicos do método. Neste ponto, pode haver uma sequência de trocas para executar o método.
5. Se a autenticação tiver sucesso e mais métodos de autenticação forem exigidos, o servidor prossegue para a etapa 3, usando um valor de sucesso parcial igual a `true`. Se a autenticação falhar, o servidor prossegue para a etapa 3, usando um valor de sucesso parcial igual a `false`.
6. Quando todos os métodos de autenticação exigidos tiverem sucesso, o servidor envia uma mensagem `SSH_MSG_USERAUTH_SUCCESS`, e o Authentication Protocol termina.

MÉTODOS DE AUTENTICAÇÃO

O servidor pode exigir um ou mais dos seguintes métodos de autenticação:

- **publickey**: os detalhes deste método dependem do algoritmo de chave pública escolhido. Basicamente, o cliente envia uma mensagem ao servidor, contendo a chave pública do cliente, com a mensagem assinada pela chave privada do cliente. Quando o servidor recebe essa mensagem, ele verifica se a chave fornecida é aceitável para autenticação e, se for, verifica se a assinatura está correta.
- **password**: o cliente envia uma mensagem contendo uma senha em texto claro, que é protegida por encriptação pelo Transport Layer Protocol.
- **hostbased**: a autenticação é realizada no hospedeiro do cliente, e não no próprio cliente. Assim, um hospedeiro que aceita vários clientes ofereceria autenticação para todos eles. Esse método funciona fazendo com que o cliente envie uma assinatura criada com a chave privada do hospedeiro do cliente. Assim, em vez de verificar diretamente a identidade do usuário, o servidor SSH verifica a identidade do hospedeiro do cliente — e então acredita no hospedeiro quando ele diz que o usuário já foi autenticado no lado do cliente.

Protocolo de Conexão

O SSH Connection Protocol trabalha em cima do SSH Transport Layer Protocol e considera que uma conexão de autenticação segura está sendo usada.² Essa conexão de autenticação segura, conhecida como **túnel**, é usada pelo Connection Protocol para multiplexar uma série de canais lógicos.

² RFC 4254, *The Secure Shell (SSH) Connection Protocol*, indica que o Connection Protocol trabalha em cima do Transport Layer Protocol e do User Authentication Protocol. A RFC 4251, *SSH Protocol Architecture*, informa que o Connection Protocol trabalha em cima do User Authentication Protocol. Na verdade, o Connection Protocol trabalha em cima do Transport Layer Protocol, mas considera que o User Authentication Protocol foi previamente invocado.

MECANISMO DO CANAL

Todos os tipos de comunicação usando SSH, como uma sessão de terminal, são aceitos usando canais separados. Qualquer um dos lados pode abrir um canal. Para cada canal, cada lado associa um número exclusivo, que não precisa ser o mesmo nas duas extremidades. Os canais têm fluxo controlado usando um mecanismo de janela. Nenhum dado pode ser enviado a um canal até que uma mensagem seja recebida para indicar que o espaço da janela está disponível.

A vida de um canal prossegue por três estágios: abrir um canal, transferir dados e fechar um canal.

Quando qualquer lado deseja **abrir um novo canal**, ele aloca um número local para o canal e depois envia uma mensagem com o formato:

byte	SSH_MSG_CHANNEL_OPEN
string	tipo de canal
uint32	canal do emissor
uint32	tamanho inicial da janela
uint32	tamanho máximo do pacote
....	em seguida, dados específicos do tipo de canal

onde uint32 significa inteiro sem sinal (unsigned integer) com 32 bits. O tipo de canal identifica a aplicação para esse canal, conforme descreveremos mais adiante. O canal do emissor é o número de canal local. O tamanho inicial da janela especifica quantos bytes de dados do canal podem ser enviados ao emissor desta mensagem sem ajustar a janela. O tamanho máximo do pacote especifica o tamanho máximo de um pacote de dados individual que pode ser enviado ao emissor. Por exemplo, pode-se querer usar pacotes menores para conexões interativas para conseguir uma melhor resposta interativa em enlaces lentos.

Se o lado remoto é capaz de abrir o canal, ele retorna uma mensagem `SSH_MSG_CHANNEL_OPEN_CONFIRMATION`, que inclui o número de canal do emissor, o número de canal do receptor e os valores de tamanho de janela e pacote para o tráfego recebido. Caso contrário, o lado remoto retorna uma mensagem `SSH_MSG_CHANNEL_OPEN_FAILURE` com um código de motivo, indicando a razão da falha.

Quando um canal é aberto, a **transferência de dados** é realizada usando uma mensagem `SSH_MSG_CHANNEL_DATA`, que inclui o número de canal do receptor e um bloco de dados. Essas mensagens, nas duas direções, podem continuar enquanto o canal estiver aberto.

Quando um dos lados quer **fechar um canal**, ele envia uma mensagem `SSH_MSG_CHANNEL_CLOSE`, que inclui o número de canal do receptor.

A Figura 17.11 oferece um exemplo de troca de mensagem do Connection Protocol.

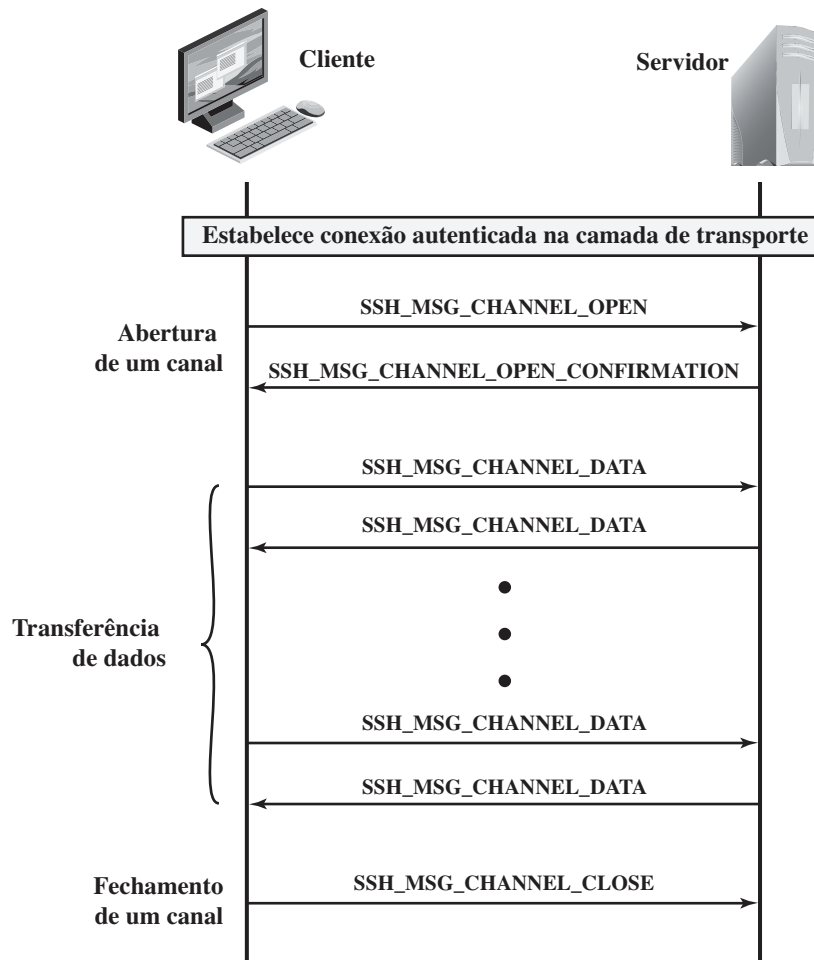
TIPOS DE CANAL

Quatro tipos de canais são reconhecidos na especificação SSH Connection Protocol.

- **session:** a execução remota de um programa. O programa pode ser um shell, uma aplicação como a de transferência de arquivos ou de e-mail, um comando do sistema ou algum subsistema embutido. Quando um canal de sessão é aberto, solicitações subsequentes são usadas para iniciar o programa remoto.
- **x11:** isto se refere ao X Window System, um sistema de software de computador e protocolo de rede que oferece uma interface gráfica com o usuário (GUI, acrônimo em inglês para *Graphical User Interface*) para computadores em rede. X permite que as aplicações sejam executadas em um servidor da rede, mas que sejam exibidas em uma máquina de desktop.
- **forwarded-tcpip:** este é o encaminhamento de porta remoto, conforme explicado na próxima subseção.
- **direct-tcpip:** este é o encaminhamento de porta local, conforme explicado na próxima subseção.

ENCAMINHAMENTO DE PORTA

Um dos recursos mais úteis do SSH é o encaminhamento de porta. Basicamente, o encaminhamento de porta oferece a capacidade de converter qualquer conexão TCP insegura em uma conexão SSH segura. Isso também é conhecido como tunelamento SSH. Neste contexto, precisamos saber o que é uma porta. Uma **porta**

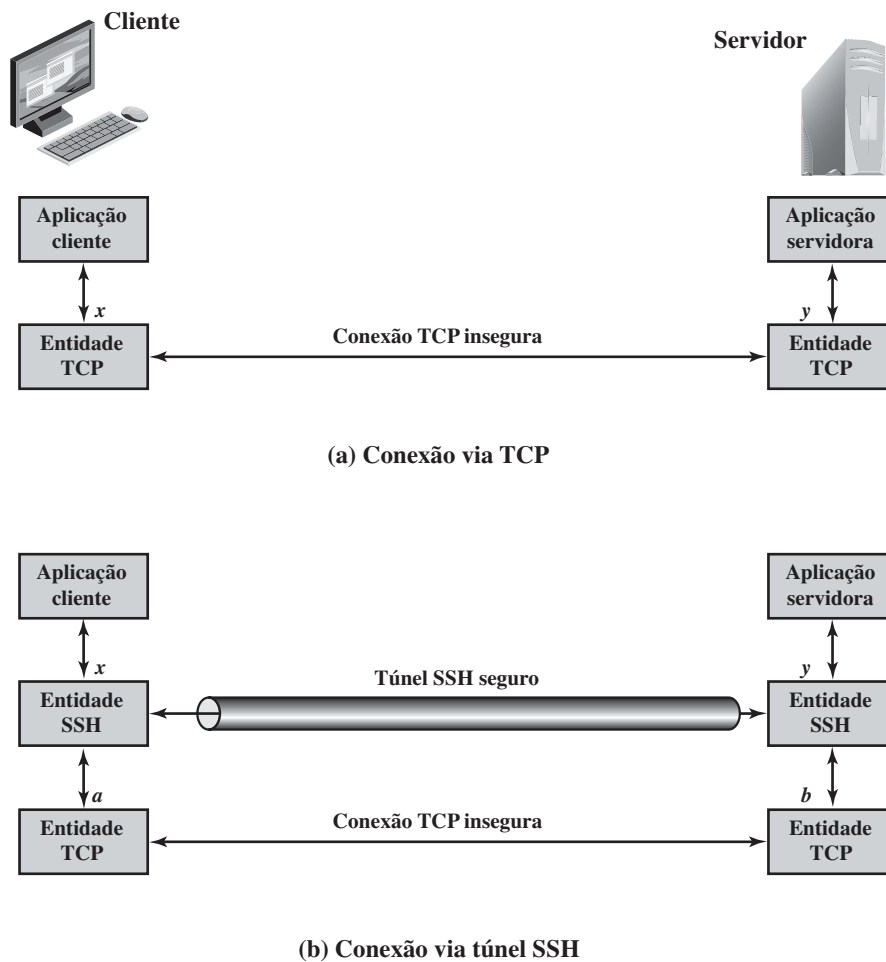
Figura 17.11 Exemplo de troca de mensagem do Connection Protocol.

é um identificador de um usuário do TCP. Assim, qualquer aplicação que é executada em cima do TCP tem um número de porta. O tráfego TCP que chega é entregue à aplicação apropriada com base no número da porta. Uma aplicação pode empregar vários números de porta. Por exemplo, para o Simple Mail Transfer Protocol (SMTP), o lado do servidor geralmente escuta na porta 25, de modo que uma solicitação SMTP que chega utiliza TCP e endereça os dados para a porta de destino 25. O TCP reconhece que esse é um endereço de servidor SMTP e direciona os dados para a aplicação de servidor SMTP.

A Figura 17.12 ilustra o conceito básico por trás do encaminhamento de porta. Temos uma aplicação cliente que é identificada pelo número de porta x e uma aplicação servidora identificada pelo número de porta y . Em algum ponto, a aplicação cliente chama a entidade TCP local e solicita uma conexão com o servidor remoto na porta y . A entidade TCP local negocia uma conexão TCP com a entidade TCP remota, de modo que a conexão liga a porta local x à porta remota y .

Para proteger essa conexão, o SSH é configurado de modo que o SSH Transport Layer Protocol estabeleça uma conexão TCP entre as entidades cliente e servidor do SSH, com os números de porta TCP a e b , respectivamente. Um túnel SSH seguro é estabelecido em cima dessa conexão TCP. O tráfego do cliente na porta x é redirecionado para a entidade SSH local e trafega pelo túnel, onde a entidade SSH remota entrega os dados à aplicação servidora na porta y . O tráfego na outra direção é redirecionado de modo semelhante.

SSH admite dois tipos de encaminhamento de porta: local e remoto. O **encaminhamento local** permite que o cliente monte um processo “sequestrador.” Este interceptará o tráfego selecionado em nível de aplicação e o redirecionará de uma conexão TCP desprotegida para um túnel SSH protegido. SSH é configurado para escutar nas portas selecionadas. SSH recebe todo o tráfego usando a porta selecionada e o envia por um túnel SSH. Na outra ponta, o servidor SSH envia o tráfego que chega à porta de destino indicada pela aplicação cliente.

Figura 17.12 Trocas de pacote da camada de transporte SSH.

O exemplo a seguir deverá ajudar a esclarecer o encaminhamento local. Suponha que você tenha um cliente de e-mail no seu desktop e o utilize para receber e-mail do seu servidor através do Post Office Protocol (POP). O número de porta atribuído para o POP3 é a porta 110. Podemos proteger esse tráfego da seguinte maneira:

1. O cliente SSH estabelece uma conexão com o servidor remoto.
2. Selecione um número de porta local não usado, digamos, 9999, e configure o SSH para aceitar o tráfego dessa porta destinado para a porta 110 no servidor.
3. O cliente SSH informa ao servidor SSH para criar uma conexão com o destino, neste caso, a porta 110 do servidor de e-mail.
4. O cliente recebe quaisquer bits enviados à porta local 9999 e os envia ao servidor dentro da sessão SSH encriptada. O servidor SSH decripta os bits que chegam e envia o texto claro à porta 110.
5. Na outra direção, o servidor SSH recebe quaisquer bits recebidos na porta 110 e os envia dentro da sessão SSH de volta ao cliente, que os decripta e os envia para o processo conectado à porta 9999.

Com o **encaminhamento remoto**, o cliente SSH do usuário atua em favor do servidor. O cliente recebe o tráfego de determinado número de porta de destino, coloca o tráfego na porta correta e o envia para o destino que o usuário escolher. Um exemplo típico de encaminhamento remoto é o seguinte. Você deseja acessar um servidor no trabalho a partir do seu computador em casa. Como o servidor no trabalho está atrás de um firewall, ele não aceitará uma solicitação SSH do seu computador em casa. Porém, do trabalho você pode estabelecer um túnel SSH usando o encaminhamento remoto. Isso envolve as seguintes etapas:

1. Do computador no trabalho, monte uma conexão SSH com o seu computador em casa. O firewall permitirá isso, pois essa é uma conexão de saída protegida.
2. Configure o servidor SSH para escutar em uma porta local, digamos, 22, e entregar os dados através da conexão SSH endereçada à porta remota, digamos, 2222.
3. Agora você pode ir para o seu computador em casa e configurar o SSH para aceitar o tráfego na porta 2222.
4. Então, você tem um túnel SSH que pode ser usado para o logon remoto com o servidor no trabalho.

17.6 LEITURA RECOMENDADA

[RESC01] é um bom tratamento detalhado sobre SSL e TLS. [BARR05] oferece um tratamento profundo do SSH. A versão original (SSH-1) do SSH foi apresentada em [YLON96].

BARR05 Barret, D.; Silverman, R.; e Byrnes, R. *SSH The Secure Shell: The Definitive Guide*. Sebastopol, CARACTERE: O'Reilly, 2005.

RESC01 Rescorla, E. *SSL and TLS: Designing and Building Secure Systems*. Reading, MA: Addison-Wesley, 2001.

YLON96 Ylonen, T. "SSH – Secure Login Connections over the Internet". *Proceedings, Sixth USENIX Security Symposium*, jul 1996.

17.7 PRINCIPAIS TERMOS, PERGUNTAS PARA REVISÃO E PROBLEMAS

Principais termos

HTTPS (HTTP over SSL)
protocolo de alerta
protocolo de especificação de mudança de cifra

protocolo de handshake
Secure Shell (SSH)
Secure Socket Layer (SSL)

segredo mestre
Transport Layer Security (TLS)

Perguntas para revisão

- 17.1 Quais são as vantagens de cada uma das três técnicas mostradas na Figura 17.1?
- 17.2 Que protocolos compreendem o SSL?
- 17.3 Qual é a diferença entre uma conexão SSL e uma sessão SSL?
- 17.4 Liste e defina resumidamente os parâmetros que definem um estado de sessão SSL.
- 17.5 Liste e defina resumidamente os parâmetros que definem uma conexão de sessão SSL.
- 17.6 Que serviços são fornecidos pelo protocolo de registro SSL?
- 17.7 Que etapas estão envolvidas na transmissão do protocolo de registro SSL?
- 17.8 Qual é a finalidade do HTTPS?
- 17.9 Para que aplicações o SSH é útil?
- 17.10 Liste e defina resumidamente os protocolos SSH.

Problemas

- 17.1 No SSL e TLS, por que existe um protocolo de especificação de mudança de cifra separado, em vez de incluir uma mensagem `change_cipher_spec` no protocolo de handshake?
- 17.2 Qual o propósito do MAC durante a troca SSL da especificação de mudança de cifra?
- 17.3 Considere as seguintes ameaças à segurança Web e descreva como cada uma é impedida por um recurso específico do SSL.

- a. Ataque criptoanalítico por força bruta: uma busca completa do espaço de chave para um algoritmo de encriptação convencional.
- b. Ataque de dicionário com texto claro conhecido: muitas mensagens terão texto claro previsível, como o comando GET do HTTP. Um atacante constrói um dicionário contendo cada encriptação possível da mensagem de texto claro conhecido. Quando uma mensagem encriptada é interceptada, o atacante apanha a parte contendo o texto claro conhecido encriptado e pesquisa o texto cifrado no dicionário. O texto cifrado deverá combinar com uma entrada que foi encriptada com a mesma chave secreta. Se houver várias combinações, cada uma delas pode ser experimentada contra o texto cifrado completo para determinar a correta. Esse ataque é especialmente eficaz contra tamanhos de chave pequenos (por exemplo, 40 bits).
- c. Ataque de replicação: mensagens de handshake SSL anteriores são replicadas.
- d. Ataque do tipo *man-in-the-middle*: um atacante se interpõe durante a troca de chave, atuando como cliente ao servidor e como servidor ao cliente.
- e. Sniffing de senha: as senhas em HTTP ou outro tráfego de aplicação são espionadas.
- f. Falsificação do IP: usa endereços IP forjados para enganar um host para aceitar dados falsos.
- g. Sequestro de IP: uma conexão ativa, autenticada, entre dois hosts é interrompida e o atacante toma o lugar de um dos hosts.
- h. Inundação de SYN: um atacante envia mensagens SYN do TCP para solicitar uma conexão, mas não responde à mensagem final para estabelecer a conexão totalmente. O módulo TCP atacado normalmente deixa a "conexão meio aberta" por alguns minutos. As mensagens SYN repetidas podem obstruir os módulos TCP.

17.4 Com base no que você aprendeu neste capítulo, é possível no SSL que o receptor reordene blocos de registro SSL que chegam fora de ordem? Nesse caso, explique como isso pode ser feito. Se não, por que não?

17.5 Para pacotes SSH, qual é a vantagem (se houver) de não incluir o MAC no escopo da encriptação de pacotes?